Value Function Approximation and DQN

Why Tabular Methods Fail?

- Up until now, we have studied algorithms like standard Q-Learning and SARSA, which rely on a Q-table. A Q-table requires a distinct row for every possible state.
- The problem with these tabular methods is that the number of possible states can be astronomically large, making it impossible to store a complete table.
- To solve this problem, we must move from *memorization* to *generalization*.
- The agent should not repeatedly learn from scratch for states with similar surroundings; it should generalize its knowledge across similar states.

Value Function Approximation

- Instead of using a lookup table, we represent the Q-value as a parameterized mathematical function:

  $$Q(s, a) \approx Q(s, a; \theta)$$

  where θ represents the parameters (or weights) of our function.

Linear vs. Non-Linear Approximators

1. **Linear Approximators:** We manually extract features from the state ($\phi(s)$) and multiply them by our weights:

   $$Q(s, a; \theta) = \theta^\top \phi(s, a)$$

   - This approach is mathematically stable and guaranteed to converge.
2. **Non-Linear Approximators (Neural Networks):** We pass the raw state into a Deep Neural Network (DNN), and it outputs the Q-values.
   - Here, the network learns and extracts the features on its own.

Gradient Descent in RL

- To train our approximator, we need to update our weights θ to minimize our errors. We use Gradient Descent for this.
- We define a loss function (which measures how far off our prediction is from the target) and update θ in the direction that reduces this loss:

  $$\theta \leftarrow \theta - \alpha \nabla_\theta \text{Loss}(\theta)$$

The DQN Loss Function

- Deep Q-Networks (DQN) train the neural network by minimizing the Mean Squared Error (MSE) between the network's current Q-value prediction and the Bellman TD Target.
- The loss function $L(\theta)$ at iteration i is defined as:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim U(D)}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i)\right)^2\right]$$

- **Where:**
  - θ_i: The weights of the primary Q-network we are currently training.
  - θ_i^-: The weights of the separate Target Network.
  - $U(D)$: Random uniform sampling from the Experience Replay Buffer (D).
  - $r+\gamma \max_{a'} Q(s', a'; \theta_i^-)$: The TD Target (what we want our network to output).
  - $Q(s, a; \theta_i)$: The current prediction from our primary network.

Why Experience Replay Helps?

- In RL, an agent generally learns from consecutive frames of an episode.
- This data is highly correlated.
- If you feed highly correlated, sequential data into a neural network, it will overfit to that specific sequence and fail to generalize.
- **Solution:** The agent stores all its experiences (s, a, r, s') in a massive dataset called the *Experience Replay Buffer*.
- During training, the network pulls random mini-batches of data from this buffer. This breaks the temporal correlation of the data, satisfying the neural network's requirement for independent and identically distributed (i.i.d.) data.

Why Target Networks Help?

- In standard Q-learning, the agent updates its value estimate based on its estimate of the next state.
- If you use the exact same neural network to calculate both the prediction and the target, the target shifts every single time the network's weights update.
- This is known as the **"moving target" problem**, and it causes feedback loops that make the network spiral out of control.
- **Solution:** DQN splits the architecture into two separate networks:
  1. The **primary network** (θ) is updated continuously.
  2. The **Target Network** (θ^-) is a frozen clone of the primary network, used only to calculate the TD target.

- Because of this separation, the targets remain stationary and stable while the primary network learns.